Week 7: Exploratory Data Analysis (EDA) and Basic Statistical Concepts

1. What is Exploratory Data Analysis (EDA)?

Core Definition

- Exploratory Data Analysis (EDA) is the process of **examining and summarizing datasets** to understand their key characteristics **before applying formal models** or algorithms.

What is EDA continued…

Key Purposes

- **Understand your data** — shape, size, and structure (rows, columns, data types).
- **Identify data quality issues** — missing values, duplicates, outliers, or inconsistent entries.
- **Explore relationships** — detect trends, correlations, and patterns between variables.
- **Guide next steps** — EDA informs decisions on feature engineering, transformations, and modeling approaches.

What is EDA continued

EDA Involves

- **Summarization:** Using descriptive statistics (mean, median, mode, variance).
- **Visualization:** Graphical tools like histograms, box plots, scatter plots, and correlation heatmaps.
- **Pattern Discovery:** Spotting relationships, anomalies, or clusters that might influence analysis.

What is EDA continued

Example

- Before predicting student performance, we first **explore the dataset**:
- Check if all subjects have the same grading scale.
- Find outliers (e.g., scores above 100).
- Identify missing attendance records.
- Visualize score distributions across gender.

Why EDA is Important

- **Detect Data Quality Issues:**
- Identify missing values, errors, duplicates, or outliers early.

- **Understand Data Distribution:**
- Learn whether variables are normal, skewed, categorical, or continuous.

- **Discover Relationships Between Variables:**
- Identify correlations, patterns, or associations that inform analysis or modelling.

- **Guide Data Cleaning and Transformation:**
- Decide on normalization, scaling, encoding, or imputing based on data behaviour.

- **Inform Modelling Choices:**
- Helps select appropriate models and features for predictive or descriptive analysis.

- **Generate Initial Insights:**
- Provides preliminary answers and helps form hypotheses for deeper analysis.

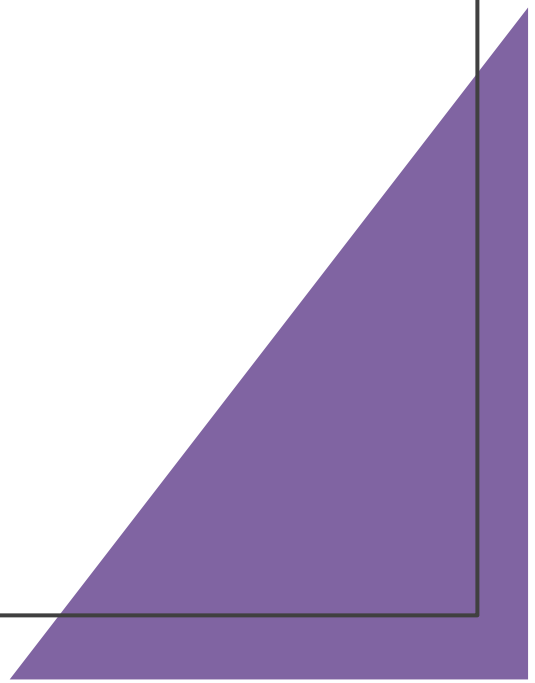
- **Communicate Findings Effectively:**
- Visualizations from EDA make complex data understandable for stakeholders.

- **Reduce Risk of Misinterpretation:**
- Helps prevent making decisions based on unclean or misunderstood data.

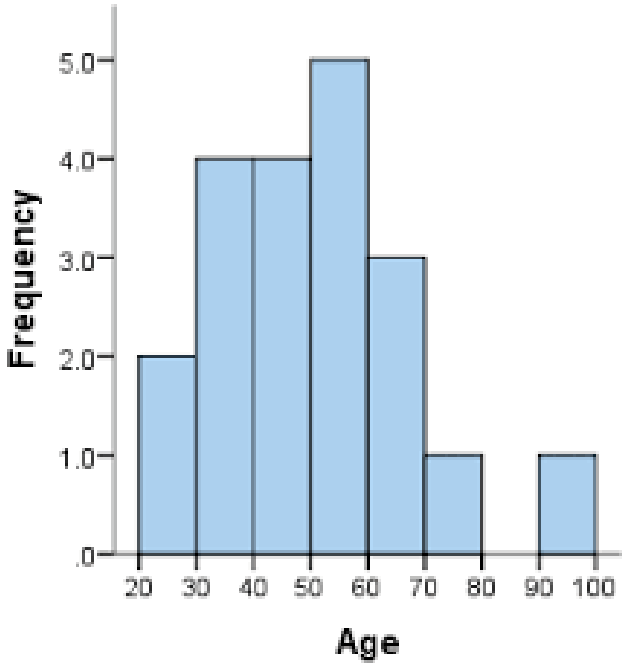
Descriptive Statistics: Understanding your data

- **Definition:** Descriptive statistics summarize and describe the main features of a dataset.

- **Purpose:**
  - Simplify complex datasets
  - Highlight patterns and trends
  - Prepare data for further analysis (EDA & modeling)

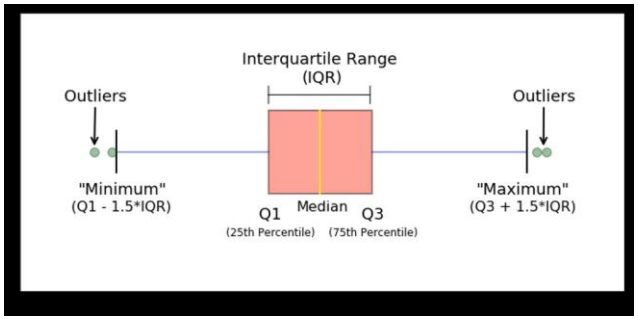


What is a Distribution?



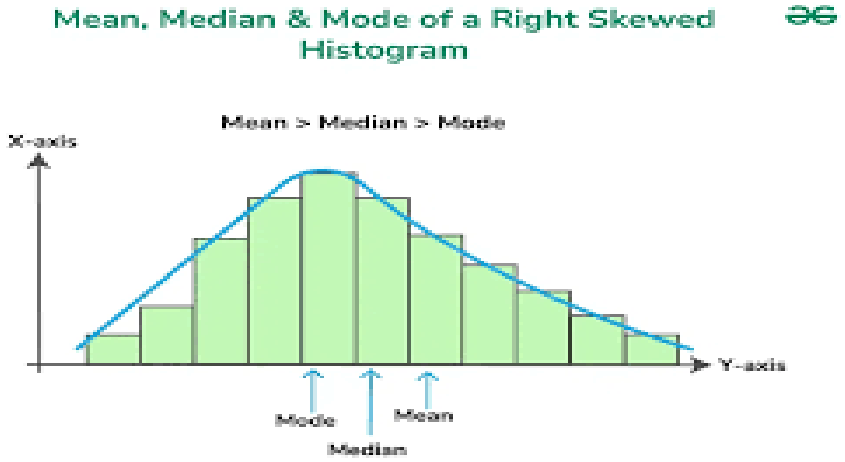
- **Definition:** A distribution shows how the values of a variable are spread or arranged.

- **Key Concepts:**
  - **Frequency:** How often each value occurs
  - **Shape:** Symmetry, skewness, and peakedness
  - **Spread:** Range of values
- **Example:** Age distribution of students in a class



Measures of Central Tendency

Statistic	What it Tells Us	Example in Python
Mean	Average value	`df['Age'].mean()`
Median	Middle value (50th percentile)	`df['Age'].median()`
Mode	Most frequent value	`df['Age'].mode()`



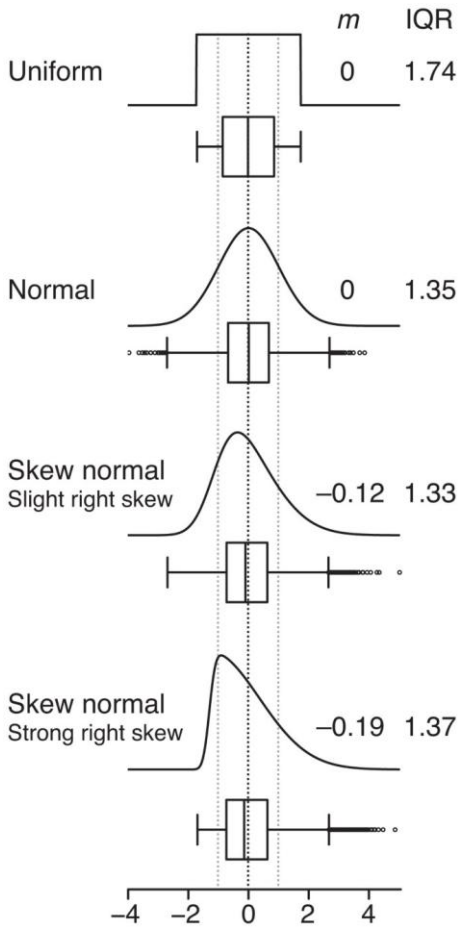
Understanding Asymmetry in a Distribution

Definition:

- Skewness measures the **asymmetry of a data distribution** around its mean.
- It tells us whether the data is **balanced or lopsided**.

Types of Skewness:

- **Positive Skew (Right Skew):**
  - Tail stretches **to the right** (towards larger values).
  - Most data values are **smaller**, with a few large outliers.
  - Mean > Median > Mode
- **Negative Skew (Left Skew):**
  - Tail stretches **to the left** (towards smaller values).
  - Most data values are **larger**, with a few small outliers.
  - Mean < Median < Mode
- **Zero Skew (Symmetrical):**
  - Data is **balanced** around the mean.
  - Mean ≈ Median ≈ Mode
  - Example: Normal distribution

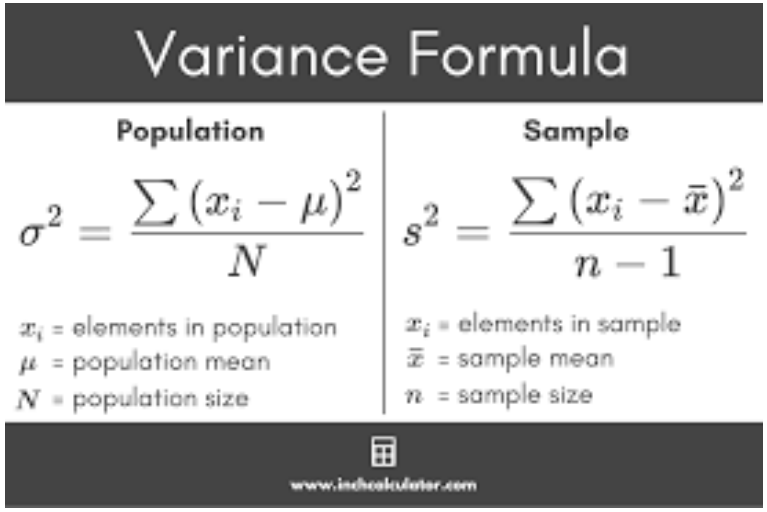


Measures of Spread

Statistic	What it Tells Us	Example in Python
Variance	Average squared deviation from the mean	`df['Age'].var()`
Standard Deviation	Square root of variance; in same units as data	`df['Age'].std()`

1. Variance

- **Definition:** Variance measures the **average squared deviation** of each data point from the mean.
- **Purpose:** Shows how much the data **varies or spreads out** from the mean.
- **Key Points:**
  - High variance → data points are widely spread.
  - Low variance → data points are close to the mean.



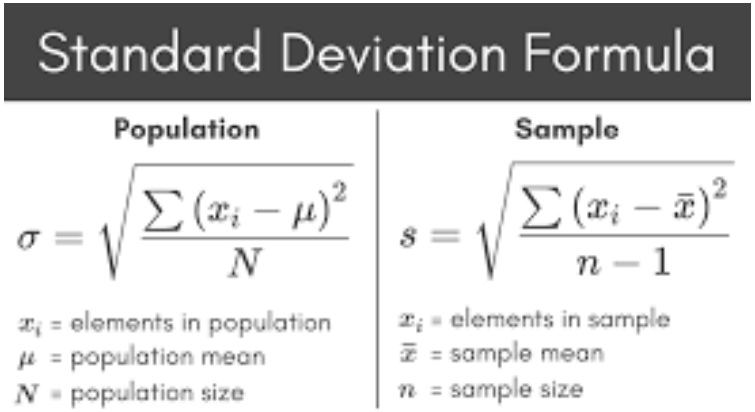
variance

```
import pandas as pd

# Example dataset
ages = [18, 20, 20, 21, 25]

# Variance
variance = pd.Series(ages).var()
# Standard deviation
std_dev = pd.Series(ages).std()

print("Variance:", variance)
print("Standard Deviation:", std_dev)
```

Measures of Spread

2. Standard Deviation (SD)

- **Definition:** Standard deviation is the **square root of variance**, giving spread in the **same units as the data**.
- **Purpose:** Makes variance easier to interpret.
- **Key Points:**
  - High SD → wide spread of values
  - Low SD → values are clustered around the mean
  - Formula:



SD

```
import pandas as pd

# Example dataset
ages = [18, 20, 20, 21, 25]

# Convert to Pandas Series
age_series = pd.Series(ages)

# Calculate standard deviation
std_dev = age_series.std()

print("Standard Deviation:", std_dev)
```

Variance vs Standard Deviation

Variance vs Standard Deviation

Definition Recap:

- **Variance:** Measures the **average squared deviation** of data points from the mean.
- **Standard Deviation (SD):** The **square root of variance**, giving spread in the **same units as the data**.

Key Relationship:

$$\text{SD} = \sqrt{Variance}$$

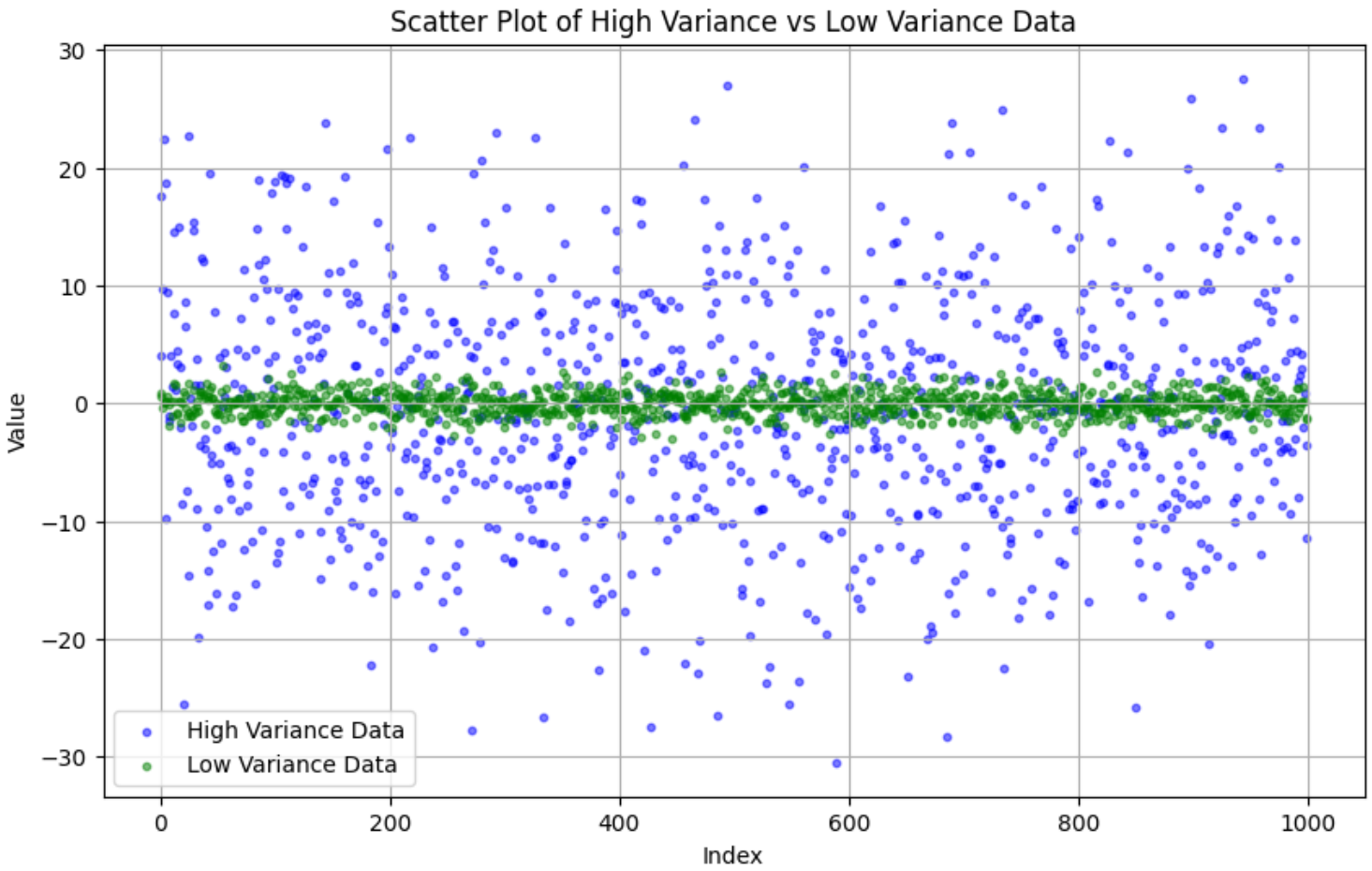
- Variance is in **squared units** (e.g., years² if measuring age).
- SD is in the **original units** (e.g., years), making it easier to interpret.

3. Intuition:

- High variance → SD will also be high
- Low variance → SD will also be low
- SD is just a **rescaled version of variance** that is more interpretable.

"Variance tells us **how spread out data is in squared terms**, SD tells us **how spread out data is in the original units**, which is usually more intuitive."

SD and Variance



Exploring Data in Pandas: Useful Functions

- Pandas is a powerful Python library for data manipulation and analysis.
- Before analysis, we need to understand the dataset's structure, content, and quality.
- These functions help us explore data quickly and efficiently.

Function	Purpose	Example / Output
`df.info()`	Shows dataset summary: column names, types, non-null counts	Gives a quick overview of dataset structure
`df.describe()`	Summarizes numeric columns: count, mean, std, min, quartiles, max	Quick summary statistics for numeric variables
`df.value_counts()`	Counts occurrences of unique values in a column	`df['Sex'].value_counts()` → {'Male': 50, 'Female': 45}
`df.isnull().sum()`	Counts missing values in each column	Identifies where data cleaning may be needed
`df.groupby('Sex')['Age'].mean()`	Aggregates data: calculates mean Age by Sex	Output: Male → 28.5, Female → 27.8

Correlation Analysis

- Correlation analysis helps us measure the strength and direction of relationships between numeric variables.

- It is a key step in EDA to understand how variables interact before modelling.

What is Correlation?

- Definition: Correlation quantifies the linear relationship between two variables.
- Range: -1 to +1
  - +1 → perfect positive correlation (as one increases, the other increases)
  - -1 → perfect negative correlation (as one increases, the other decreases)
  - 0 → no linear correlation
- Use Case: Helps identify predictor variables and potential multicollinearity.

Pandas and Seaborn for Correlation

- **1. Using Pandas:**

```
# Compute correlation matrix
corr_matrix = df.corr()
print(corr_matrix)
```

- **2. Visualizing with Seaborn Heatmap:**

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

Visualizing Distributions: Histograms and Boxplots

•Visualizations are a key part of **EDA**, helping us understand **how data is distributed** and **identify patterns or outliers**.

•Two common ways to visualize numeric data: **Histograms** and **Boxplots**.

1. Histogram

Definition:

- A histogram displays the **frequency distribution** of a numeric variable by grouping values into bins.
- **KDE (Kernel Density Estimate)** shows a smooth estimate of the distribution.

Python Example:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df['Age'], bins=20, kde=True)
plt.title("Histogram of Age")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

Interpretation:

- Shows **where most data points are concentrated**.
- Reveals **skewness** and potential **outliers**.
- Helps decide whether to **transform** variables (e.g., log-transform skewed data).

2. Boxplot

Definition:

- A boxplot (or whisker plot) shows **median, quartiles, range, and outliers**.
- Useful for **comparing distributions across groups**.

- **Python Example:**

```
sns.boxplot(x='Survived', y='Age', data=df)
plt.title("Boxplot of Age by Survival")
plt.show()
```

Interpretation:

- Box → interquartile range (IQR)
- Line inside box → median
- Whiskers → range of typical data
- Dots outside whiskers → outliers
- Helps **compare distributions between categories** (e.g., Survived = 0 vs 1)

Scatter and Pair Plots: Visualizing Relationships

•Beyond distributions, we often want to **explore relationships between two or more variables**.

•Scatter plots and pair plots are powerful tools for this purpose.

1. Scatter Plot

Definition:

- A scatter plot shows the relationship between two numeric variables.
- Adding a hue allows us to distinguish groups (e.g., categorical outcomes).

Python Example:

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.scatterplot(x='Age', y='Fare', hue='Survived', data=df)
plt.title("Scatter Plot of Age vs Fare by Survival")
plt.xlabel("Age")
plt.ylabel("Fare")
plt.show()
```

Interpretation:

Pattern: Do higher fares correlate with age?
Group differences: Are survivors clustered differently from non-survivors?
Outliers: Spot unusual points that may affect analysis.

2. Pair Plot (Scatterplot Matrix)

Definition:

- A pair plot shows scatter plots for every combination of selected numeric variables.
- Often includes histograms or KDE on the diagonal.
- Color-coded by a categorical variable to compare groups.

Python Example:

```
sns.pairplot(df[['Age','Fare','Survived']], hue='Survived')
plt.show()
```

Interpretation:

- Quickly observe relationships between multiple variables.
- Identify linear or non-linear trends, clusters, and potential correlations.
- Diagonal plots show variable distributions for each group.

Grouping and Aggregation in Pandas

- Often in EDA, we want to **summarize numeric data by categories**.
- Grouping and aggregation help us **compute statistics for subsets of data**, such as averages, counts, or sums.
- This is key for **comparing patterns across groups**.

1. Basic Grouping
Python Example:

```
# Mean Fare by Passenger Class
df.groupby('Pclass')['Fare'].mean()
```

Interpretation:

- Groups data by **Passenger Class** (Pclass) and calculates the **average fare** for each class.
- Useful for **understanding differences between categories**.

Example Output:

```
Pclass
1    87.5
2    21.0
3        13.0
```

- Shows that **first-class passengers paid higher fares** on average.

2. Grouping by Multiple Columns

Python Example:

```
# Mean survival rate by Sex and Pclass
df.groupby(['Sex','Pclass'])['Survived'].mean()
```

Interpretation:

- Groups data by **Sex** and **Passenger Class**, then calculates the **average survival rate**.
- Reveals **interaction effects** between categories.
- Example Output:

```
Sex     Pclass
female  1        0.97
        2        0.92
        3        0.50
male    1        0.36
        2        0.16
        3        0.13
```

- Shows **females had higher survival rates**, especially in higher classes.

Probability and Normal Distribution

•Probability is the **likelihood of an event occurring**.

•Many statistical methods assume that **data follows a normal distribution** (bell-shaped curve).

•Understanding distributions helps in **modeling, hypothesis testing, and inference**.

1. What is a Normal Distribution?

- A **symmetric, bell-shaped distribution** where most values cluster around the mean.

- **Characteristics:**
  - Mean = Median = Mode
  - About **68% of data** lies within 1 standard deviation of the mean
  - About **95% of data** lies within 2 standard deviations
  - About **99.7% of data** lies within 3 standard deviations (Empirical Rule)

2. Python Example: Simulating Normal Data

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Generate random normal data
data = np.random.normal(0, 1, 1000)

# Plot histogram with KDE
sns.histplot(data, kde=True)
plt.title("Histogram of Normally Distributed Data")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Explanation:

- np.random.normal(mean, std, size) generates synthetic normally distributed data.
- Histogram with KDE visualizes the bell-shaped distribution.

3. Interpretation

•“Normal distributions are fundamental in statistics for **probability calculations and inferential methods**.”

•“Many statistical tests (t-test, ANOVA, regression) assume **normally distributed residuals**.”

•“Visual inspection with histograms or KDE helps verify this assumption.”

•“Real data may not be perfectly normal; small deviations are often acceptable.”

9. Project 1: Titanic Dataset EDA

- 1. Load dataset: sns.load_dataset('titanic')
- 2. Explore survival by Sex, Age, Pclass
- 3. Visualize relationships using Seaborn plots

Code example

```
# Step 1: Import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 2: Load dataset
df = sns.load_dataset('titanic')

# Quick look at the dataset
print(df.head())
print(df.info())

# Step 3: Explore survival by Sex, Age, and Pclass
# Survival counts by Sex
print(df.groupby('sex')['survived'].value_counts(normalize=True))

# Survival rate by Pclass
print(df.groupby('pclass')['survived'].mean())

# Survival rate by Sex and Pclass
print(df.groupby(['sex','pclass'])['survived'].mean())

# Step 4: Visualizations

# 4a: Bar plot - Survival by Sex
sns.countplot(x='sex', hue='survived', data=df)
plt.title("Survival Counts by Sex")
plt.show()

# 4b: Boxplot - Age distribution by Survival
sns.boxplot(x='survived', y='age', data=df)
plt.title("Age Distribution by Survival")
plt.show()

# 4c: Histogram - Age distribution
sns.histplot(df['age'], bins=20, kde=True)
plt.title("Histogram of Age")
plt.show()

# 4d: Scatter plot - Age vs Fare colored by Survival
sns.scatterplot(x='age', y='fare', hue='survived', data=df)
plt.title("Age vs Fare by Survival")
plt.show()
```

10. Wrap-Up

1. Purpose of EDA:

- Understand your data before modeling.
- Detect errors, missing values, outliers, and unusual patterns.
- Inform data cleaning, feature selection, and transformations.

2. Key Tools and Techniques:

- **Descriptive Statistics:** Mean, Median, Mode, Variance, Standard Deviation, Skewness.
- **Data Exploration in Pandas:** df.info(), df.describe(), value_counts(), isnull().sum(), groupby().
- **Visualizations:**
  - Histograms & KDE → distributions
  - Boxplots → spread & outliers
  - Scatter & Pair plots → relationships
  - Heatmaps → correlations

3. Practical Insights:

- Patterns and relationships help **generate hypotheses**.
- Visualizations make data **intuitive and interpretable**.
- EDA is **iterative**: explore → clean → visualize → summarize → repeat.

4. Next Steps:

- Use insights from EDA to **prepare for modeling or reporting**.
- Always document findings to **support decision-making**.
- **Visual Suggestion:**
- A flow diagram:
- Raw Data → EDA (Summary Stats + Visuals) → Cleaned Data → Modeling / Insights